After you call `Build()` on the `WebApplicationBuilder` instance, you can't register any more services or change your logging configuration; the services defined for the `WebApplication` instance are set in stone. The next step is defining how your application responds to HTTP requests.

3.7.2 Defining how requests are handled with middleware and endpoints

After registering your services with the IoC container on `WebApplicationBuilder` and doing any further customization, you create a `WebApplication` instance. You can do three main things with the `WebApplication` instance:

- Add middleware to the pipeline.
- Map endpoints that generate a response for a request.
- Run the application by calling `Run()`.

As I described previously, middleware consists of small components that execute in sequence when the application receives an HTTP request. They can perform a host of functions, such as logging, identifying the current user for a request, serving static files, and handling errors. Middleware is typically added to `WebApplication` by calling `Use*` extension methods. In listing 3.4, I showed an example of adding the `HttpLoggingMiddleware` to the middleware pipeline conditionally by calling `UseHttpLogging()`:

```
if (app.Environment.IsDevelopment())
{
    app.UseHttpLogging();
}
```

We added only a single piece of middleware to the pipeline in this example, but when you're adding multiple pieces of middleware, the order of the `Use*` calls